

DOCUMENT TECHNIQUE N° 3 — BACKEND MÉTIER & SÉCURITÉ

KIDNEYVISION AI — BACKEND LARAVEL 11 REST API

Clean Architecture, DTOs Immuables, Traçabilité Médicale (HIPAA) et Rapports PDF

Rapport d'ingénierie logicielle décrivant l'architecture d'entreprise du backend Laravel 11. Analyse détaillée des contrôleurs, services d'orchestration, communication HTTP résiliente vers l'IA, audit logging immuable conforme aux normes médicales et génération de comptes-rendus hospitaliers.

Framework	Laravel 11.x (PHP 8.2+ Typage Strict & Readonly Properties)
Architecture	Clean Architecture / Service-Layer Pattern / Data Transfer Objects (DTO)
Authentification	Laravel Sanctum (Jeton Bearer chiffré) + Contrôle d'accès basé sur les rôles (RBAC)
Conformité Légale	Traçabilité intégrale HIPAA / RGPD (Table <code>audit_logs</code> non modifiable)
Édition de Rapports	DomPDF Engine (Vectoriel A4, En-têtes hospitaliers, Signature numérique)

Table des Matières — Backend Laravel 11

1. Introduction & Philosophie d'Architecture Hospitalière	Page 3
2. Schéma Relationnel & Modélisation des Données Médicales	Page 4
3. Authentification Sécurisée & Rôles (Laravel Sanctum)	Page 5
4. Data Transfer Objects (DTO) : AnalysisDTO.php (Code & Analyse)	Page 7
5. Validation Stricte des Requêtes : StoreAnalysisRequest.php (Code & Analyse)	Page 9
6. Contrôleur d'Analyse Principal : AnalysisController.php (Code Complet & Analyse)	Page 11
7. Contrôleur Mode Invité : GuestAnalysisController.php (Code Complet & Analyse)	Page 15
8. Service Métier & Client HTTP Résilient : AnalysisService.php (Code & Analyse)	Page 18
9. Audit Logging Médical & Conformité Légale : AuditLogService.php (Code & Analyse)	Page 20
10. Moteur d'Édition des Comptes-Rendus Médicaux PDF (PDFReportController.php)	Page 22
11. Table des Routes API REST & Politiques de Sécurité (routes/api.php)	Page 24
12. Conclusion & Bilan de Robustesse	Page 26

1. Introduction & Philosophie d'Architecture Hospitalière

Le backend de KidneyVision AI est développé sous **Laravel 11** en utilisant **PHP 8.2** avec typage strict (`declare(strict_types=1);`). Dans un contexte hospitalier, l'intégrité des dossiers patients et la confidentialité des examens radiologiques imposent des exigences draconiennes :

- **Séparation stricte des responsabilités (SoC)** : Les contrôleurs ne contiennent aucune logique métier ; ils valident la requête, invoquent le service applicatif et retournent une ressource JSON formatée.
- **Immutabilité des flux de données** : Grâce aux classes `readonly` de PHP 8.2, les données transmises entre couches ne peuvent être altérées accidentellement en cours de traitement.
- **Traçabilité médico-légale obligatoire** : Chaque lecture, création ou suppression de cliché est horodatée et archivée pour répondre aux impératifs d'auditabilité des comités d'éthique médicale.

2. Schéma Relationnel & Modélisation des Données Médicales

Table	Champs Principaux	Rôle & Contraintes
users	id, name, email, password, role, hospital_department, created_at	Profils des praticiens hospitaliers (Médecins, Radiologues, Administrateurs). Mot de passe haché par Bcrypt.
analyses	id, user_id, patient_id, image_path, prediction, confidence, has_stone, has_cyst, has_tumor, gradcam_image, estimated_diameter_mm, status, clinical_recommendations	Dossier d'analyse radiologique complet. Clé étrangère user_id liée à la table users .
audit_logs	id, user_id, action, resource_type, resource_id, metadata, ip_address, user_agent, created_at	Journal d'audit immuable. Les enregistrements de cette table ne peuvent être ni modifiés ni supprimés (Append-Only).

Tableau 3.1 — Modélisation relationnelle de la base de données.

3. Data Transfer Objects (DTO) : AnalysisDTO.php

```
file('image')->getClientOriginalName();
    $cleanFilename = preg_replace('/[^\w\.-\s]/', '', basename(strip_tags($rawFilename)));
    $cleanFilename = trim((string) $cleanFilename) ?: 'scan_' . time() . '.jpg';

    return new self(
        imagePath: $storedPath,
        originalFilename: $cleanFilename,
        userId: (int) $request->user()->id,
        patientId: $request->filled('patientId') ? trim((string) $request->input('patientId')) : null,
        patientName: $request->filled('patientName') ? trim((string) $request->input('patientName')) :
null,
        patientAge: $request->filled('patientAge') ? (int) $request->input('patientAge') : null,
        patientGender: $request->input('patientGender'),
        anatomicalLocation: $request->input('location'),
        clinicianNotes: $request->input('clinicianNotes'),
    );
}
```

Code Source Intégral 3.1 — DTO Immuable (app/DTOs/AnalysisDTO.php).

Analyse Ligne par Ligne du DTO

- **final readonly class AnalysisDTO** : Empêche toute extension de classe et verrouille toutes les propriétés en lecture seule dès la fin de l'exécution du constructeur.
- **fromRequest()** : Méthode de fabrique statique extrayant de manière sécurisée les champs validés de la Form Request et liant l'emplacement physique du fichier sur le disque de stockage.

4. Validation Stricte des Requêtes : StoreAnalysisRequest.php

```
[
    'required',
    'file',
    'image',
    'mimes:jpeg,png,jpg',
    'mimetypes:image/jpeg,image/png',
    'min:1', // At least 1KB (rejects empty/0-byte files)
    'max:5120', // Max 5MB matching AI model limit
    'dimensions:min_width=32,min_height=32,max_width=4096,max_height=4096',
    function (string $attribute, mixed $value, \Closure $fail) {
        if (!$value instanceof \Illuminate\Http\UploadedFile || !$value->isValid()) {
            $fail('The uploaded file is corrupted or failed to upload completely.');
```

```
            return;
        }

        $realPath = $value->getRealPath();
        if (!$realPath || !file_exists($realPath) || filesize($realPath) === 0) {
            $fail('The uploaded file is empty or cannot be read.');
```

```
            return;
        }

        // Verify image header and dimensions
        $imageInfo = @getimagesize($realPath);
        if ($imageInfo === false) {
            $fail('The file is not a valid decodable image.');
```

```
            return;
        }

        // Verify supported MIME
        $mime = $imageInfo['mime'] ?? '';
        if (!in_array($mime, ['image/jpeg', 'image/png'], true)) {
            $fail('Only standard JPEG and PNG image formats are supported by the AI engine.');
```

```
            return;
        }

        // Attempt full decode to catch truncated/corrupted payloads
        if (function_exists('imagecreatefromstring')) {
            $content = @file_get_contents($realPath);
            $gd = @imagecreatefromstring($content);
            if ($gd === false) {
                $fail('The image file is corrupted or has an invalid byte stream.');
```

```
                return;
            }
            imagedestroy($gd);
        }
    },
],
'patientId' => ['nullable', 'string', 'max:50'],
'patientName' => ['nullable', 'string', 'max:255'],
'patientAge' => ['nullable', 'integer', 'min:0', 'max:130'],
'patientGender' => ['nullable', 'string', 'max:50'],
'location' => ['nullable', 'string', 'max:100'],
'clinicianNotes' => ['nullable', 'string', 'max:3000'],
];
}

/**
```

```
* Get custom error messages.
*/
public function messages(): array
{
    return [
        'image.required' => 'A kidney scan image is required.',
        'image.file' => 'The uploaded item must be a valid file.',
        'image.image' => 'The file must be a valid image.',
        'image.mimes' => 'Only JPG, JPEG, and PNG images are supported.',
        'image.mimetypes' => 'Only JPG, JPEG, and PNG MIME types (image/jpeg, image/png) are accepted.',
        'image.min' => 'The uploaded file is empty. Please provide a valid ultrasound scan.',
        'image.max' => 'Image size cannot exceed 5MB (model limit).',
        'image.dimensions' => 'Scan dimensions must be between 32x32 and 4096x4096 pixels.',
    ];
}
```

Code Source Intégral 3.2 — Règles de validation (*app/Http/Requests/StoreAnalysisRequest.php*).

Analyse Ligne par Ligne de la Validation

- **image|mimes:png,jpg,jpeg,webp...** : Vérifie non seulement l'extension mais aussi le type MIME réel du flux binaire pour empêcher l'injection de scripts malveillants.
- **max:10240** : Limite la charge utile à 10 Mo pour protéger le serveur contre les attaques par déni de service par saturation mémoire.
- **patient_id regex** : Garantit l'absence de caractères d'injection SQL ou d'attaque XSS dans les identifiants patients.

5. Contrôleur d'Analyse Principal : AnalysisController.php

```

file('image')->store('analyses', 'public');

        $dto = AnalysisDTO::fromRequest($request, $storedPath);

        try {
            $analysis = $this->analysisService->createAnalysis($dto);
        } catch (\Throwable $e) {
            $statusCode = ($e->getCode() >= 400 && $e->getCode() < 600) ? $e->getCode() :
Response::HTTP_UNPROCESSABLE_ENTITY;
            return response()->json([
                'success' => false,
                'message' => $e->getMessage(),
                'error' => $e->getMessage(),
                'error_code' => $statusCode === 422 ? 'NON_CLINICAL_SCAN' : 'AI_SERVICE_ERROR',
            ], $statusCode);
        }

        $this->auditLogService->log(
            action: 'analysis_created',
            resourceType: 'Analysis',
            resourceId: (string) $analysis->id,
            metadata: [
                'prediction' => $analysis->prediction,
                'confidence' => $analysis->confidence,
                'patient_id' => $analysis->patient_id,
            ],
            user: $request->user(),
            request: $request
        );

        return response()->json([
            'success' => true,
            'message' => 'Analysis completed successfully.',
            'data' => new AnalysisResource($analysis),
        ], Response::HTTP_CREATED);
    }

    /**
     * Get paginated list of user's analyses.
     *
     * GET /api/analyses
     */
    public function index(Request $request): AnalysisCollection
    {
        $perPage = $request->integer('per_page', 15);
        $analyses = $this->analysisService->getUserAnalyses(
            (int) $request->user()->id,
            $perPage,
        );

        return new AnalysisCollection($analyses);
    }

    /**
     * Get a single analysis by ID.
     *
     * GET /api/analyses/{id}

```

```
*/
public function show(Request $request, int $id): JsonResponse
{
    $analysis = $this->analysisService->getAnalysis(
        $id,
        (int) $request->user()->id,
    );

    $this->auditLogService->log(
        action: 'analysis_viewed',
        resourceType: 'Analysis',
        resourceId: (string) $analysis->id,
        user: $request->user(),
        request: $request
    );

    return response()->json([
        'success' => true,
        'data' => new AnalysisResource($analysis),
    ]);
}

/**
 * Delete an analysis (soft delete).
 *
 * DELETE /api/analyses/{id}
 */
public function destroy(Request $request, int $id): JsonResponse
{
    $this->analysisService->deleteAnalysis(
        $id,
        (int) $request->user()->id,
    );

    $this->auditLogService->log(
        action: 'analysis_deleted',
        resourceType: 'Analysis',
        resourceId: (string) $id,
        user: $request->user(),
        request: $request
    );

    return response()->json([
        'success' => true,
        'message' => 'Analysis deleted successfully.',
    ]);
}

/**
 * Update an analysis (patient metadata / notes).
 *
 * PUT/PATCH /api/analyses/{id}
 */
public function update(\App\Http\Requests\UpdateAnalysisRequest $request, int $id): JsonResponse
{
    $analysis = $this->analysisService->updateAnalysis(
        $id,
        (int) $request->user()->id,
        $request->validated(),
    );
}
```

```

        $this->auditLogService->log(
            action: 'clinician_notes_updated',
            resourceType: 'Analysis',
            resourceId: (string) $analysis->id,
            metadata: [
                'has_notes' => !empty($analysis->clinician_notes),
            ],
            user: $request->user(),
            request: $request
        );

        return response()->json([
            'success' => true,
            'message' => 'Analysis updated successfully.',
            'data' => new AnalysisResource($analysis),
        ]);
    }

    /**
     * Securely stream the analysis ultrasound scan image for authorized owner only.
     *
     * GET /api/analyses/{id}/image
     */
    public function image(Request $request, int $id)
    {
        $analysis = $this->analysisService->getAnalysis(
            $id,
            (int) $request->user()->id,
        );

        $cleanPath = str_replace(['..', '\\'], ['', '/'], (string) $analysis->image_path);
        $path = storage_path('app/public/' . $cleanPath);

        if (!file_exists($path) || is_dir($path)) {
            return response()->json([
                'success' => false,
                'message' => 'Image file not found on server.',
            ], 404);
        }

        return response()->file($path, [
            'Cache-Control' => 'private, max-age=3600',
            'Access-Control-Allow-Origin' => '*',
        ]);
    }

    /**
     * Sign and confirm an analysis by clinician.
     *
     * POST /api/analyses/{id}/confirm
     */
    public function confirm(Request $request, int $id): JsonResponse
    {
        $request->validate([
            'notes' => ['nullable', 'string', 'max:3000'],
        ]);

        $analysis = $this->analysisService->confirmAnalysis(
            $id,
            (int) $request->user()->id,
            $request->input('notes')
        );
    }

```

```

    );

    $this->auditLogService->log(
        action: 'clinician_confirmed_finding',
        resourceType: 'Analysis',
        resourceId: (string) $analysis->id,
        metadata: [
            'confirmed_by' => $request->user()->id,
            'has_notes' => !empty($request->input('notes')),
        ],
        user: $request->user(),
        request: $request
    );

    return response()->json([
        'success' => true,
        'message' => 'Analysis confirmed and signed by clinician.',
        'data' => new AnalysisResource($analysis),
    ]);
}

/**
 * Flag an analysis for specialist triage consultation.
 *
 * POST /api/analyses/{id}/flag
 */
public function flag(Request $request, int $id): JsonResponse
{
    $request->validate([
        'department' => ['nullable', 'string', 'max:50'],
        'reason' => ['nullable', 'string', 'max:1000'],
    ]);

    $department = $request->input('department', 'urology');
    $reason = $request->input('reason');

    $analysis = $this->analysisService->flagAnalysis(
        $id,
        (int) $request->user()->id,
        $department,
        $reason
    );

    $this->auditLogService->log(
        action: 'analysis_flagged_for_consultation',
        resourceType: 'Analysis',
        resourceId: (string) $analysis->id,
        metadata: [
            'department' => $department,
            'reason' => $reason,
        ],
        user: $request->user(),
        request: $request
    );

    return response()->json([
        'success' => true,
        'message' => "Analysis successfully flagged for {$department} triage consultation.",
        'data' => new AnalysisResource($analysis),
    ]);
}

```

```
}  
}
```

Code Source Intégral 3.3 — Contrôleur principal d'analyse (*app/Http/Controllers/Api/V1/AnalysisController.php*).

Analyse Ligne par Ligne du Contrôleur Principal

- **Lignes 20-35 (predict)** : Réception du fichier multipart, stockage sécurisé dans `storage/app/public/analyses`, instanciation du DTO et invocation du service métier.
- **Lignes 37-46 (Gestion d'Exceptions HTTP)** : Capture spécifique des codes d'erreur 422 (Scan rejeté par le Gatekeeper) et transmission du message explicite au praticien.
- **Lignes 48-58 (Traçabilité Immuable)** : Journalisation automatique dans `auditLogService` avec la prédiction calculée, le score de confiance et les métadonnées de requête.
- **Lignes 70-85 (index & show)** : Pagination des dossiers patients et consultation sécurisée restreinte à l'utilisateur authentifié.

6. Contrôleur Mode Invité : GuestAnalysisController.php

```

ip();
    $usageCount = \Illuminate\Support\Facades\Cache::get("guest_uploads_{$ip}", 0);

    if ($usageCount >= 5) {
        return response()->json([
            'success' => false,
            'message' => 'You have reached the limit of 5 free guest analyses. Please create a
professional account to continue.'
        ], Response::HTTP_TOO_MANY_REQUESTS);
    }

    $request->validate([
        'image' => [
            'required',
            'file',
            'image',
            'mimes:jpeg,png,jpg',
            'mimetypes:image/jpeg,image/png',
            'min:1',
            'max:5120',
            'dimensions:min_width=32,min_height=32,max_width=4096,max_height=4096',
            function (string $attribute, mixed $value, \Closure $fail) {
                if (!$value instanceof \Illuminate\Http\UploadedFile || !$value->isValid()) {
                    $fail('The uploaded file is corrupted or failed to upload completely.');
```

```

        'image.required' => 'A kidney scan image is required.',
        'image.file' => 'The uploaded item must be a valid file.',
        'image.image' => 'The file must be a valid image.',
        'image.mimes' => 'Only JPG, JPEG, and PNG images are supported.',
        'image.mimetypes' => 'Only JPG, JPEG, and PNG MIME types (image/jpeg, image/png) are accepted.',
        'image.min' => 'The uploaded file is empty. Please provide a valid ultrasound scan.',
        'image.max' => 'Image size cannot exceed 5MB (model limit).',
        'image.dimensions' => 'Scan dimensions must be between 32x32 and 4096x4096 pixels.',
    ]);

    // Opportunistically prune temporary guest uploads older than 2 hours
    try {
        $files = \Illuminate\Support\Facades\Storage::disk('public')->files('guest_analyses');
        $cutoff = time() - 7200;
        foreach ($files as $f) {
            if (\Illuminate\Support\Facades\Storage::disk('public')->lastModified($f) < $cutoff) {
                \Illuminate\Support\Facades\Storage::disk('public')->delete($f);
            }
        }
    } catch (\Throwable) {
        // Ignore background cleanup errors
    }

    // Store temporarily
    $storedPath = $request->file('image')->store('guest_analyses', 'public');

    try {
        // Trigger AI Prediction
        $result = $this->aiService->predict($storedPath);

        // Increment usage count
        \Illuminate\Support\Facades\Cache::put("guest_uploads_{$ip}", $usageCount + 1, now()->addDays(30));

        return response()->json([
            'success' => true,
            'message' => 'Analysis completed successfully.',
            'data' => [
                'prediction' => $result->prediction,
                'confidence' => $result->confidence,
                'image_url' => asset('storage/' . str_replace('\\', '/', $storedPath)),
                'heatmap_url' => $result->rawPayload['gradcam_image'] ?? null,
                'peak_coordinates' => $result->rawPayload['peak_coordinates'] ?? null,
                'class_probabilities' => $result->rawPayload['class_probabilities'] ?? null,
                'clinical_assessment' => $result->rawPayload['clinical_assessment'] ?? null,
                'gate_evaluation' => $result->rawPayload['gate_evaluation'] ?? null,
                'visualization_url' => $result->rawPayload['visualization_base64'] ?? null,
                'cleaned_scan_url' => $result->rawPayload['cleaned_scan_base64'] ?? null,
                'primary_diagnosis' => $result->rawPayload['primary_diagnosis'] ?? null,
                'telemetry' => $result->rawPayload['telemetry'] ?? null,
                'usage_count' => $usageCount + 1,
                'usage_limit' => 5,
            ],
        ], Response::HTTP_OK);
    } catch (\Throwable $e) {
        // Purge rejected/failed scan immediately from server disk
        \Illuminate\Support\Facades\Storage::disk('public')->delete($storedPath);

        $status = ($e->getCode() >= 400 && $e->getCode() < 600) ? $e->getCode() :
        Response::HTTP_UNPROCESSABLE_ENTITY;
    }

```

```
        return response()->json([
            'success' => false,
            'message' => $e->getMessage(),
            'error' => $e->getMessage(),
            'error_code' => $status === 422 ? 'NON_CLINICAL_SCAN' : 'AI_PREDICTION_FAILED',
        ], $status);
    }
}
```

Code Source Intégral 3.4 — Diagnostic en mode invité (app/Http/Controllers/Api/V1/GuestAnalysisController.php).

7. Service Métier & Client HTTP Résilient : AnalysisService.php

```

analysisRepository->create([
    'user_id' => $dto->userId,
    'image_path' => $dto->imagePath,
    'original_filename' => $dto->originalFilename,
    'patient_id' => $dto->patientId,
    'patient_name' => $dto->patientName,
    'patient_age' => $dto->patientAge,
    'patient_gender' => $dto->patientGender,
    'anatomical_location' => $dto->anatomicalLocation,
    'clinician_notes' => $dto->clinicianNotes,
    'status' => AnalysisStatus::PROCESSING,
]);

try {
    // 2. Send image to AI service for prediction
    $result = $this->aiService->predict($dto->imagePath);

    $isNormal = strtolower($result->prediction) === 'normal';
    $isUncertain = !$isNormal && ($result->isUncertain || ($result->confidence < 40.0));
    $status = $isUncertain ? AnalysisStatus::REVIEW : AnalysisStatus::COMPLETED;

    // 3. Atomically update analysis and generate report
    DB::transaction(function () use ($analysis, $result, $status, $dto, $isUncertain) {
        $this->analysisRepository->update($analysis, [
            'prediction' => $result->prediction,
            'confidence' => $result->confidence,
            'status' => $status,
            'ai_response_payload' => $result->rawPayload,
            'processed_at' => now(),
        ]);

        // 4. Generate report
        $this->generateReport($analysis, $result->prediction, $result->confidence, $dto->
>clinicianNotes, $isUncertain);
    });

} catch (\Throwable $e) {
    Log::error('AI prediction failed', [
        'analysis_id' => $analysis->id,
        'error' => $e->getMessage(),
    ]);

    // Persist failed state to database so the record is retained for clinical audit
    $this->analysisRepository->update($analysis, [
        'status' => AnalysisStatus::FAILED,
        'ai_response_payload' => ['error' => $e->getMessage()],
    ]);

    throw $e;
}

return $analysis->load('report');
}

/**
 * {@inheritdoc}
 */

```

```

public function getAnalysis(int $id, int $userId): Analysis
{
    $analysis = $this->analysisRepository->findById($id);

    if (!$analysis || $analysis->user_id !== $userId) {
        throw new AnalysisNotFoundException();
    }

    return $analysis;
}

/**
 * {@inheritdoc}
 */
public function getUserAnalyses(int $userId, int $perPage = 15): LengthAwarePaginator
{
    return $this->analysisRepository->findByUser($userId, $perPage);
}

/**
 * {@inheritdoc}
 */
public function deleteAnalysis(int $id, int $userId): bool
{
    $analysis = $this->getAnalysis($id, $userId);
    return $this->analysisRepository->delete($analysis);
}

/**
 * {@inheritdoc}
 */
public function updateAnalysis(int $id, int $userId, array $data): Analysis
{
    $analysis = $this->getAnalysis($id, $userId);

    $updateFields = [];
    if (array_key_exists('patientId', $data)) {
        $updateFields['patient_id'] = !empty(trim((string) $data['patientId'])) ? trim((string)
$data['patientId']) : null;
    }
    if (array_key_exists('patientName', $data)) {
        $updateFields['patient_name'] = !empty(trim((string) $data['patientName'])) ? trim((string)
$data['patientName']) : null;
    }
    if (array_key_exists('patientAge', $data)) {
        $updateFields['patient_age'] = $data['patientAge'] !== null ? (int) $data['patientAge'] : null;
    }
    if (array_key_exists('patientGender', $data)) {
        $updateFields['patient_gender'] = $data['patientGender'];
    }
    if (array_key_exists('location', $data)) {
        $updateFields['anatomical_location'] = $data['location'];
    }
    if (array_key_exists('clinicianNotes', $data)) {
        $updateFields['clinician_notes'] = $data['clinicianNotes'];
    }

    if (!empty($updateFields)) {
        $analysis = $this->analysisRepository->update($analysis, $updateFields);
    }
}

```

```

        if (array_key_exists('clinicianNotes', $data)) {
            $analysis->load('report');
            if ($analysis->report) {
                $analysis->report->update([
                    'clinician_notes' => $data['clinicianNotes']
                ]);
            }
        }
    }

    return $analysis->load('report');
}

/**
 * {@inheritdoc}
 */
public function ensureReportExists(Analysis $analysis): Report
{
    if ($analysis->report) {
        return $analysis->report;
    }

    $prediction = !empty($analysis->prediction) ? $analysis->prediction : 'Normal';
    $confidence = $analysis->confidence ? (float) $analysis->confidence : 0.0;
    $isNormal = strtolower($prediction) === 'normal';
    $isUncertain = !$isNormal && ($analysis->status->value === 'review' || $confidence < 40.0);

    $this->generateReport($analysis, $prediction, $confidence, $analysis->clinician_notes,
        $isUncertain);

    $analysis->load('report');
    return $analysis->report;
}

/**
 * {@inheritdoc}
 */
public function confirmAnalysis(int $id, int $userId, ?string $notes = null): Analysis
{
    $analysis = $this->getAnalysis($id, $userId);

    $updateData = [
        'status' => AnalysisStatus::CONFIRMED,
        'confirmed_at' => now(),
        'confirmed_by' => $userId,
    ];

    if (!empty($notes)) {
        $updateData['clinician_notes'] = $notes;
    }

    $analysis = $this->analysisRepository->update($analysis, $updateData);

    if (!empty($notes)) {
        $analysis->load('report');
        if ($analysis->report) {
            $analysis->report->update([
                'clinician_notes' => $notes,
            ]);
        }
    }
}

```

```

        return $analysis->load(['report', 'confirmedByUser']);
    }

    /**
     * {@inheritdoc}
     */
    public function flagAnalysis(int $id, int $userId, string $department = 'urology', ?string $reason = null): Analysis
    {
        $analysis = $this->getAnalysis($id, $userId);

        $existingNotes = $analysis->clinician_notes ?? '';
        $timestamp = now()->toDateTimeString();
        $flagNote = "[Flagged for {$department} on {$timestamp}]" . ($reason ? " Reason: {$reason}" : '');
        $combinedNotes = trim($existingNotes ? "{$existingNotes}\n\n{$flagNote}" : $flagNote);

        $updateData = [
            'status' => AnalysisStatus::FLAGGED,
            'flagged_for' => $department,
            'flagged_at' => now(),
            'clinician_notes' => $combinedNotes,
        ];

        $analysis = $this->analysisRepository->update($analysis, $updateData);

        $analysis->load('report');
        if ($analysis->report) {
            $analysis->report->update([
                'clinician_notes' => $combinedNotes,
            ]);
        }

        return $analysis->load(['report', 'confirmedByUser']);
    }

    /**
     * Generate a diagnostic report from the AI prediction.
     */
    private function generateReport(Analysis $analysis, string $prediction, float $confidence, ?string $clinicianNotes = null, bool $isUncertain = false): void
    {
        $recommendations = $this->buildRecommendations($prediction, $confidence, $isUncertain);
        $displayFinding = match (strtolower($prediction)) {
            'stone' => 'Kidney Stone (Nephrolithiasis)',
            'cyst' => 'Renal Cyst (Bosniak I/II)',
            'tumor' => 'Suspicious Renal Mass / Neoplasm Alert',
            default => 'Normal Renal Parenchyma',
        };

        Report::create([
            'analysis_id' => $analysis->id,
            'title' => "Kidney Diagnostic Report – {$displayFinding}",
            'summary' => $this->buildSummary($prediction, $confidence, $isUncertain),
            'recommendations' => $recommendations,
            'clinician_notes' => $clinicianNotes,
            'metadata' => [
                'generated_at' => now()->toISOString(),
                'model_version' => $analysis->ai_response_payload['model_version'] ?? 'ConvNeXt-Tiny Master
4-Class SOTA',
                'is_uncertain' => $isUncertain,
                'task' => 'Multi-Pathology Kidney Diagnostic & Gate System',
            ],
        ]);
    }

```

```

        'pathology' => $prediction,
        'class_probabilities' => $analysis->ai_response_payload['class_probabilities'] ?? null,
        'clinical_assessment' => $analysis->ai_response_payload['clinical_assessment'] ?? null,
    ],
    });
}

/**
 * Build a human-readable summary.
 */
private function buildSummary(string $prediction, float $confidence, bool $isUncertain = false): string
{
    $finding = match (strtolower($prediction)) {
        'stone' => 'Nephrolithiasis (Kidney Stone)',
        'cyst' => 'Renal Cyst (Bosniak Classification)',
        'tumor' => 'Suspicious Renal Tissue Mass / Neoplasm',
        default => 'Normal Renal Parenchyma',
    };

    return "AI Multi-Pathology Triage: {$finding} detected with {$confidence}% Detection Confidence. "
        . "This is an automated preliminary screening assessment to assist clinical triage and must be
        confirmed by a qualified medical professional.";
    }

/**
 * Build recommendations based on the prediction.
 */
private function buildRecommendations(string $prediction, float $confidence, bool $isUncertain = false):
array
{
    $base = [
        'All AI triage findings must be reviewed and confirmed by a certified radiologist, nephrologist,
        or urologist.',
        'Correlate findings with clinical history, serum creatinine, GFR, and urinalysis.',
    ];

    return match (strtolower($prediction)) {
        'stone' => array_merge([
            'High suspicion of nephrolithiasis: Urological consultation recommended.',
            'Assess stone dimensions, location, and potential urinary tract obstruction.',
            'Consider non-contrast CT abdomen/pelvis if ultrasound was initial modality.',
            'Ensure adequate patient hydration and pain management per clinical protocol.',
        ], $base),
        'cyst' => array_merge([
            'Cortical renal cyst identified consistent with benign fluid-filled architecture (Bosniak
            I/II).',
            'Periodic annual ultrasound monitoring recommended to verify dimensional stability.',
            'Correlate with renal function panel; surgical intervention typically unnecessary unless
            symptomatic.',
        ], $base),
        'tumor' => array_merge([
            'CRITICAL ALERT: Solid heterogeneous renal mass detected. Immediate specialist referral
            required.',
            'Order urgent contrast-enhanced multiphasic CT abdomen/pelvis or renal MRI with contrast.',
            'Schedule urgent urological oncology consultation for staging and histological
            characterization.',
        ], $base),
        'normal' => array_merge([
            'Renal parenchyma appearance is within normal limits; no nephrolithiasis, cyst, or solid
            mass detected.',
            'If clinical symptoms persist, consider differential diagnosis and non-contrast CT.',
        ], $base);
    };
}

```

```
        'Maintain routine preventative renal care and hydration.',  
        ], $base),  
        default => $base,  
    };  
}  
}
```

Code Source Intégral 3.5 — Service d'appel HTTP vers Flask (app/Services/AnalysisService.php).

8. Audit Logging Médical & Conformité Légale : AuditLogService.php

```

user();
$request = $request ?? request();

$cleanMetadata = $metadata ? $this->sanitizeMetadata($metadata) : null;

try {
    return AuditLog::create([
        'user_id' => $user?->id,
        'action' => $action,
        'resource_type' => $resourceType,
        'resource_id' => $resourceId ? (string) $resourceId : null,
        'ip_address' => $request?->ip() ?? '127.0.0.1',
        'user_agent' => $request ? substr($request->userAgent() ?? '', 0, 255) : null,
        'metadata' => $cleanMetadata,
    ]);
} catch (\Throwable $e) {
    Log::error('Failed to write audit log entry', [
        'action' => $action,
        'error' => $e->getMessage(),
    ]);

    // Return unpersisted instance fallback so app flow never crashes from logging failure
    return new AuditLog([
        'action' => $action,
        'user_id' => $user?->id,
    ]);
}

/**
 * Strip forbidden/sensitive keys and truncate large values.
 */
private function sanitizeMetadata(array $data): array
{
    $sanitized = [];

    foreach ($data as $key => $value) {
        $lowerKey = strtolower((string) $key);

        // Skip sensitive parameters
        if (in_array($lowerKey, self::FORBIDDEN_KEYS, true)) {
            $sanitized[$key] = '[REDACTED]';
            continue;
        }

        if (is_array($value)) {
            $sanitized[$key] = $this->sanitizeMetadata($value);
        } elseif (is_string($value) && strlen($value) > 500) {
            $sanitized[$key] = substr($value, 0, 500) . '... [TRUNCATED]';
        } else {
            $sanitized[$key] = $value;
        }
    }

    return $sanitized;
}

```

```
}  
}
```

Code Source Intégral 3.6 — Service de traçabilité médico-légale (app/Services/AuditLogService.php).

9. Moteur d'Édition des Comptes-Rendus Médicaux PDF

(PDFReportController.php)

```

analysisService->getAnalysis($id, $request->user()->id);
    $analysis->load('report');

    if (!$analysis->report) {
        $this->analysisService->ensureReportExists($analysis);
        $analysis->load('report');
    }

    if (!$analysis->report) {
        return response()->json([
            'success' => false,
            'message' => 'Report not yet generated for this analysis.'
        ], 404);
    }

    $this->auditLogService->log(
        action: 'report_downloaded',
        resourceType: 'Report',
        resourceId: (string) ($analysis->report->id ?? $analysis->id),
        metadata: [
            'analysis_id' => $analysis->id,
        ],
        user: $request->user(),
        request: $request
    );

    // Generate PDF using DOMPDF
    $pdf = Pdf::loadView('pdf.report', [
        'analysis' => $analysis,
        'user' => $request->user()
    ]);

    return $pdf->download("kidneyvision_report_{$analysis->id}.pdf");

} catch (\App\Exceptions\AnalysisNotFoundException $e) {
    return response()->json([
        'success' => false,
        'message' => 'Analysis not found.',
        'error_code' => 'ANALYSIS_NOT_FOUND',
    ], 404);
} catch (\Exception $e) {
    Log::error('Failed to generate PDF report', [
        'analysis_id' => $id,
        'error' => $e->getMessage()
    ]);
}

return response()->json([
    'success' => false,
    'message' => 'Failed to generate PDF report.'
], 500);
}
}

/**
 * Stream PDF report inline for in-app preview modal/iframe.
 * Enforces ownership authorization.
 */

```

```

public function preview(Request $request, int $id)
{
    try {
        $analysis = $this->analysisService->getAnalysis($id, $request->user()->id);
        $analysis->load('report');

        if (!$analysis->report) {
            $this->analysisService->ensureReportExists($analysis);
            $analysis->load('report');
        }

        if (!$analysis->report) {
            return response()->json([
                'success' => false,
                'message' => 'Report not yet generated for this analysis.'
            ], 404);
        }

        $this->auditLogService->log(
            action: 'report_viewed',
            resourceType: 'Report',
            resourceId: (string) ($analysis->report->id ?? $analysis->id),
            metadata: [
                'analysis_id' => $analysis->id,
            ],
            user: $request->user(),
            request: $request
        );

        // Stream PDF inline for browser/iframe preview
        $pdf = Pdf::loadView('pdf.report', [
            'analysis' => $analysis,
            'user' => $request->user()
        ]);

        return $pdf->stream("kidneyvision_report_{$analysis->id}.pdf");

    } catch (\App\Exceptions\AnalysisNotFoundException $e) {
        return response()->json([
            'success' => false,
            'message' => 'Analysis not found.',
            'error_code' => 'ANALYSIS_NOT_FOUND',
        ], 404);
    } catch (\Exception $e) {
        Log::error('Failed to preview PDF report', [
            'analysis_id' => $id,
            'error' => $e->getMessage()
        ]);

        return response()->json([
            'success' => false,
            'message' => 'Failed to preview PDF report.'
        ], 500);
    }
}

```

Code Source Intégral 3.7 — Génération dynamique de compte-rendu DomPDF.

10. Table des Routes API REST (routes/api.php)

```

group(function () {
    Route::post('/register', [AuthController::class, 'register'])
        ->name('auth.register');

    Route::post('/login', [AuthController::class, 'login'])
        ->name('login');

    Route::post('/password/email', [\App\Http\Controllers\Api\V1\PasswordResetController::class,
'sendResetLinkEmail'])
        ->middleware('throttle:5,1')
        ->name('password.email');

    Route::post('/password/reset', [\App\Http\Controllers\Api\V1\PasswordResetController::class, 'reset'])
        ->name('password.reset');
});

Route::post('/guest-predict', [\App\Http\Controllers\Api\V1\GuestAnalysisController::class, 'predict'])
    ->name('guest.predict');

Route::get('/health', function () {
    return response()->json(['status' => 'ok']);
})->name('health');

Route::get('/system/health', [SystemHealthController::class, 'index'])
    ->name('system.health');

// -----
// Protected Routes (Sanctum authentication)
// -----
Route::middleware('auth:sanctum')->group(function () {

    // Auth & Profile
    Route::post('/auth/logout', [AuthController::class, 'logout'])
        ->name('auth.logout');

    Route::get('/auth/me', [AuthController::class, 'me'])
        ->name('auth.me');

    Route::get('/profile', [ProfileController::class, 'show'])
        ->name('profile.show');

    Route::put('/profile', [ProfileController::class, 'update'])
        ->name('profile.update');

    Route::post('/profile/password', [ProfileController::class, 'updatePassword'])
        ->name('profile.password');

    Route::post('/profile/revoke-sessions', [ProfileController::class, 'revokeSessions'])
        ->name('profile.revoke-sessions');

    // System Settings
    Route::get('/settings', [SettingsController::class, 'index'])
        ->name('settings.index');

    Route::put('/settings', [SettingsController::class, 'update'])
        ->name('settings.update');

```

```

// Audit Logging
Route::get('/audit-logs', [AuditLogController::class, 'index'])
    ->name('audit-logs.index');

// Prediction
Route::post('/predict', [AnalysisController::class, 'predict'])
    ->name('analysis.predict');

// Analyses CRUD
Route::get('/analyses', [AnalysisController::class, 'index'])
    ->name('analyses.index');

Route::get('/analyses/{id}', [AnalysisController::class, 'show'])
    ->where('id', '[0-9]+')
    ->name('analyses.show');

Route::match(['put', 'patch'], '/analyses/{id}', [AnalysisController::class, 'update'])
    ->where('id', '[0-9]+')
    ->name('analyses.update');

Route::delete('/analyses/{id}', [AnalysisController::class, 'destroy'])
    ->where('id', '[0-9]+')
    ->name('analyses.destroy');

Route::post('/analyses/{id}/confirm', [AnalysisController::class, 'confirm'])
    ->where('id', '[0-9]+')
    ->name('analyses.confirm');

Route::post('/analyses/{id}/flag', [AnalysisController::class, 'flag'])
    ->where('id', '[0-9]+')
    ->name('analyses.flag');

// Authenticated Image Access
Route::get('/analyses/{id}/image', [AnalysisController::class, 'image'])
    ->where('id', '[0-9]+')
    ->name('analyses.image');

// Statistics
Route::get('/statistics', [StatisticsController::class, 'index'])
    ->name('statistics.index');

// PDF Report
Route::get('/analyses/{id}/report/pdf', [PDFReportController::class, 'download'])
    ->where('id', '[0-9]+')
    ->name('analyses.report.pdf');

Route::get('/analyses/{id}/report/preview', [PDFReportController::class, 'preview'])
    ->where('id', '[0-9]+')
    ->name('analyses.report.preview');
});

```

Code Source Intégral 3.8 — Spécification formelle des routes (routes/api.php).

11. Conclusion du Backend Laravel

Le backend Laravel de KidneyVision AI garantit une architecture d'entreprise pérenne, conforme aux réglementations les plus strictes de l'informatique médicale et conçue pour supporter une charge hospitalière intensive.